Food Delivery Data Analysis

Project Overview

This project perform Exploratory Data Analysis(EDA) on food delivery order data to understand order patterns, vendor revenue,delivery performance,and customer raings.

Objectives

- Analyze order distribution across cities.
- Identify top-performing vendors by order value.
- Study Delivery time and distance patterns.
- investigate delivery delays and cancellations.
- Explore monthly order value trends.

Tools Used

Python,Pandas,Seaborn,Matplotib,Jupyter Notebook

Dataset

3,000 records and 17 original columns

EDA - Exploratory Data Analysis

it is used for analysing data before importing file

1.Importing library

In [1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

2. Here file is Imported and Created DataFrame

In [2]:
```
try:
    df=pd.read_csv(r"C:\Users\ITFACT\Downloads\delivery_orders_dataset.csv")
except FileNotFoundError:
    print("plz check file path")
else:
    print("file loaded")
```

```
file loaded
```

3. Inspecting file

3.1 Here is file info and Dataframe(checked Datatypes also)

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 17 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   order_id           3000 non-null   str    
 1   order_date         3000 non-null   str    
 2   order_placed_time  3000 non-null   str    
 3   day_of_week        3000 non-null   str    
 4   city               3000 non-null   str    
 5   vendor_id          3000 non-null   str    
 6   vendor_name        3000 non-null   str    
 7   order_type         3000 non-null   str    
 8   driver_id          3000 non-null   str    
 9   distance_km        3000 non-null   float64
 10  weather_condition  3000 non-null   str    
 11  is_peak_hour       3000 non-null   int64  
 12  order_value_inr    3000 non-null   float64
 13  delivery_time_min  2913 non-null   float64
 14  is_delayed         3000 non-null   int64  
 15  is_cancelled       3000 non-null   int64  
 16  customer_rating    2913 non-null   float64
dtypes: float64(4), int64(3), str(10)
memory usage: 398.6 KB
```

3.2 Here if Duplicate found it will remove and file will run

In [5]:
```
# this code is use to automatically remove duplicate is found or if not then run fi
if df.duplicated().sum()>0:
    print("duplicate found",df.duplicated().sum())
    df.drop_duplicates(inplace=True)
```

```
else:
    print("no duplicate found")
```

```
no duplicate found
```

3.3 Here checking is there null values are present

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
order_id              0
order_date            0
order_placed_time     0
day_of_week           0
city                  0
vendor_id             0
vendor_name           0
order_type            0
driver_id             0
distance_km           0
weather_condition     0
is_peak_hour          0
order_value_inr       0
delivery_time_min    87
is_delayed            0
is_cancelled          0
customer_rating      87
dtype: int64
```

3.4 Here is Total count of Rows and Column

In [7]:
```
size=df.shape
print("total row  and column is=", size)
```

```
total row  and column is= (3000, 17)
```

3.1.1 Changing Datatype str -> date time

In [8]:
```
df["order_date"]=pd.to_datetime(df["order_date"])
```

In [9]:
```
df["order_date"]
```

Out[9]:
```
0      2026-01-03
1      2026-02-07
2      2026-04-08
3      2026-04-20
4      2026-03-10
          ...
2995   2026-06-22
2996   2026-05-15
2997   2026-02-05
2998   2026-04-22
2999   2026-05-25
Name: order_date, Length: 3000, dtype: datetime64[us]
```

4 univariate analysis

1. show total order count
2. show total vendors registed on platform
3. show avg order price
4. show min,max,and avg delivery
5. show avg distance in km

In [10]:
```
total_order=df["order_id"].count()
print("total order",total_order)
```

```
total order 3000
```

In [11]:
```
total_vender = df["vendor_id"].count()
print("total register platform",total_vender)
```

```
total register platform 3000
```

In [12]:
```
avg_price=df["order_value_inr"].mean()
print("avg price is",avg_price)
```

```
avg price is 763.0920500000001
```

In [15]:
```
min_delivary_time =df["delivery_time_min"].min()
print(" min time is",min_delivary_time)
```

```
min time is 6.9
```

In [16]:
```
max_delivary_time =df["delivery_time_min"].max()
print(" min time is",max_delivary_time)
```

```
min time is 109.7
```

In [17]:
```
avg_delivary_time =df["delivery_time_min"].mean()
print(" min time is",avg_delivary_time)
```

```
min time is 31.126364572605564
```

In [18]:
```
avg_time_distance=df["distance_km"].mean().round()
print("avg distance time is",avg_time_distance)
```

```
avg distance time is 3.0
```

5 Adding new columns

In [19]:
```
def delivery_remark(time):
    if time>60:
        return "very late"
    elif time>30:
        return "late"
    elif time>10:
        return "on time"
    else:
        return "very fast"
```

In [20]:
```
delivery_remark(30)
```

Out[20]:
```
'on time'
```

In [21]:
```
def record(km):
    if km>10:
        return "long distance"
    elif km>5:
        return "short distance"
    else:
        return "near"
```

In [22]:
```
record(3)
```

Out[22]:
```
'near'
```

In [23]:
```
df["distance_record"]=df["distance_km"].map(record)
```

In [24]:
```
df.groupby("distance_record")["order_id"].count().reset_index()
```

Out[24]:

	distance_record	order_id
0	long distance	27
1	near	2498
2	short distance	475

Data Visualization in python

6.1 city by order count

In [25]:
```
city_order=df.groupby("city")["order_id"].count().sort_values().reset_index()
```

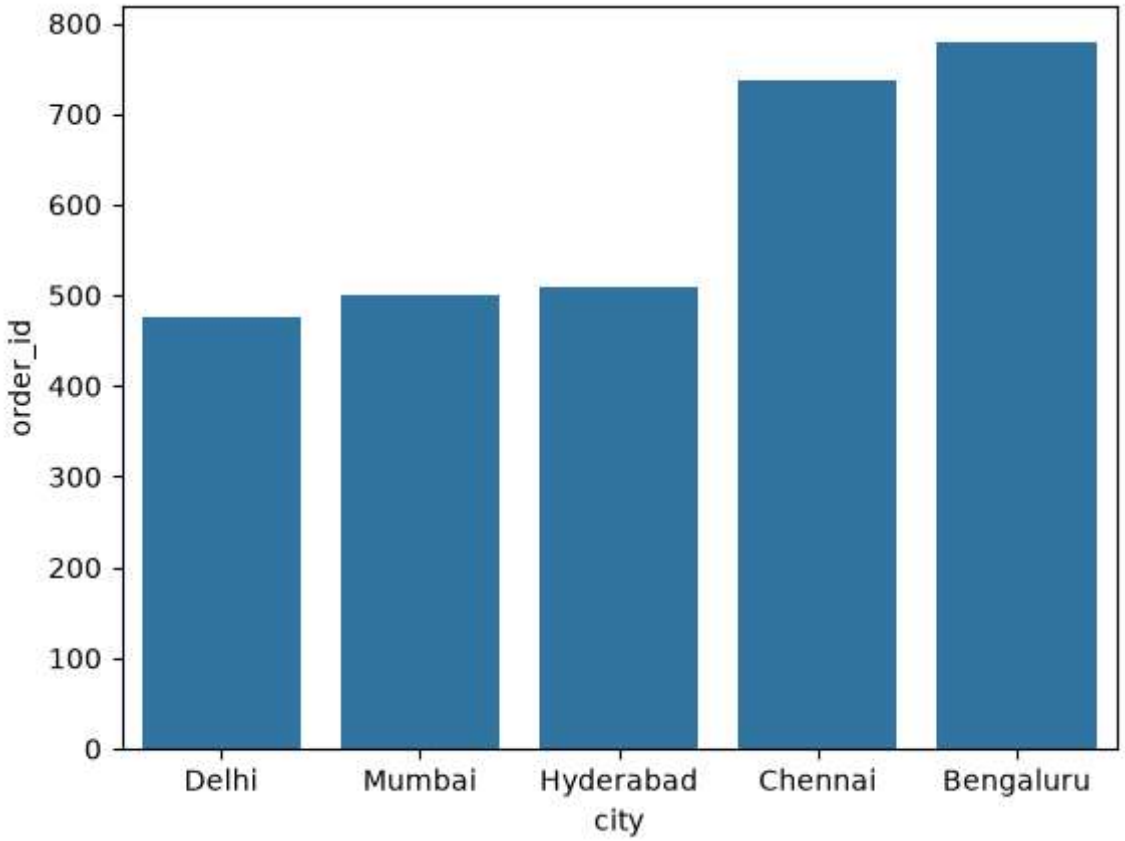
In [26]:
```
city_order
```

Out[26]:

	city	order_id
0	Delhi	475
1	Mumbai	501
2	Hyderabad	509
3	Chennai	736
4	Bengaluru	779

In [27]:
```
sns.barplot(x="city",y="order_id",data=city_order)
```

Out[27]:
```
<Axes: xlabel='city', ylabel='order_id'>
```

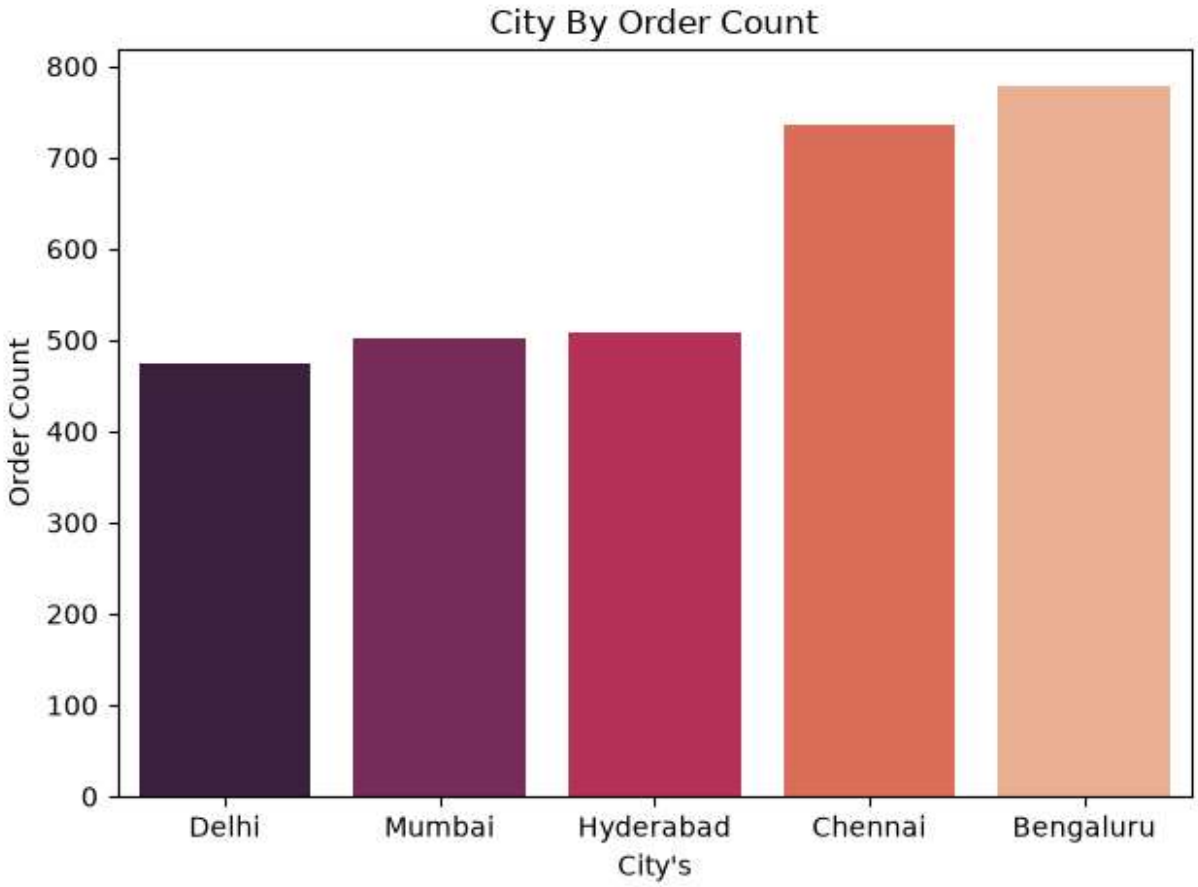


In [28]:
```
sns.barplot(x="city",y="order_id",data=city_order,palette="rocket")
plt.title("City By Order Count") # give Title for graph
plt.xlabel("City's") #give x axie lable name
plt.ylabel("Order Count") #give y axie lable name
plt.tight_layout() # this commond give border to graph
plt.savefig("city by order count.png") # to save graph
plt.show() # it shows the visual
```

```
C:\Users\ITFACT\AppData\Local\Temp\ipykernel_9892\3402944608.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

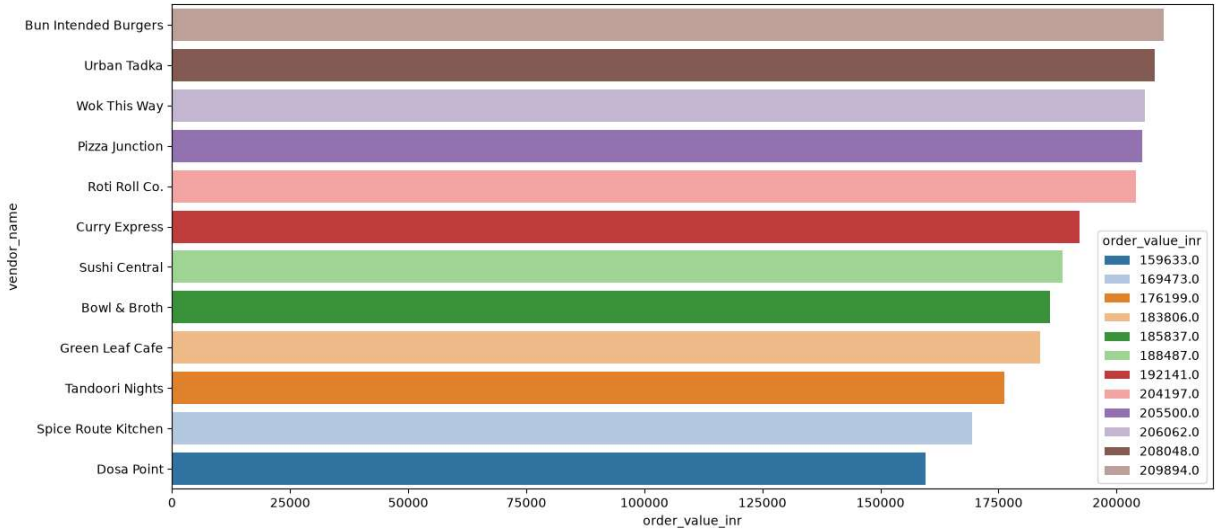
  sns.barplot(x="city",y="order_id",data=city_order,palette="rocket")
```



6.2 vendor by revenue

In [29]:
```
vendor_revenue=df.groupby("vendor_name")["order_value_inr"].sum().round().sort_valu
```

In [30]:
```
plt.figure(figsize=(15,7))
sns.barplot(y="vendor_name",x="order_value_inr",data=vendor_revenue,palette="tab20"
plt.show()
```



6.3 weather data

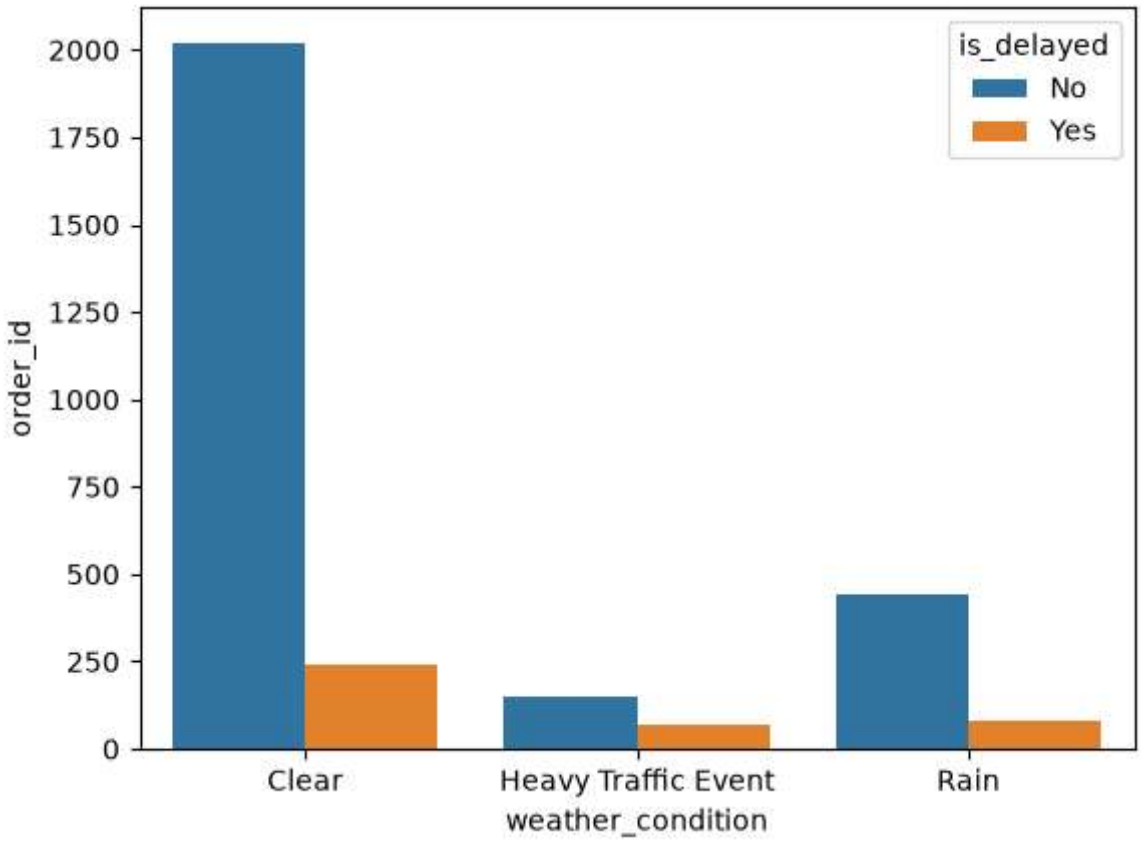
In [31]:
```
df["is_delayed"]=df["is_delayed"].replace(1,"Yes").replace(0,"No")
```

In [32]:
```
df["is_cancelled"]=df["is_cancelled"].replace(1,"Yes").replace(0,"No")
```

In [33]:
```
weather_data=df.groupby(["weather_condition","is_delayed"])["order_id"].count().res
```

In [34]:
```
sns.barplot(x="weather_condition",y="order_id",data=weather_data,hue="is_delayed")
```

Out[34]:
```
<Axes: xlabel='weather_condition', ylabel='order_id'>
```



In [35]:
```
df.columns
```

Out[35]:
```
Index(['order_id', 'order_date', 'order_placed_time', 'day_of_week', 'city',
       'vendor_id', 'vendor_name', 'order_type', 'driver_id', 'distance_km',
       'weather_condition', 'is_peak_hour', 'order_value_inr',
       'delivery_time_min', 'is_delayed', 'is_cancelled', 'customer_rating',
       'distance_record'],
      dtype='str')
```

6.4 delivery distance vs delivery time

- scatterplot use when yoy have to show indiviual thing

In [36]:
```
df.columns
```

Out[36]:
```
Index(['order_id', 'order_date', 'order_placed_time', 'day_of_week', 'city',
       'vendor_id', 'vendor_name', 'order_type', 'driver_id', 'distance_km',
       'weather_condition', 'is_peak_hour', 'order_value_inr',
       'delivery_time_min', 'is_delayed', 'is_cancelled', 'customer_rating',
       'distance_record'],
      dtype='str')
```

In [37]:
```
sns.scatterplot(x="distance_km",y="delivery_time_min",data=df,hue="weather_conditio
plt.title(" delivery distance vs delivery time")
```

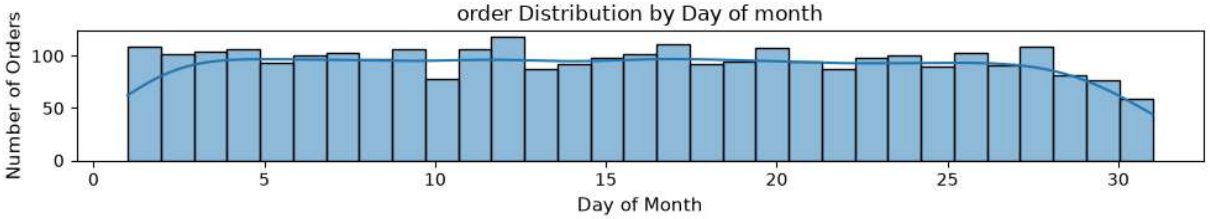
Out[37]:
```
Text(0.5, 1.0, ' delivery distance vs delivery time')
```



In [38]:
```
df["day"]=df["order_date"].dt.day # extracting day month year from date column
```

6.5 Day by Day order count

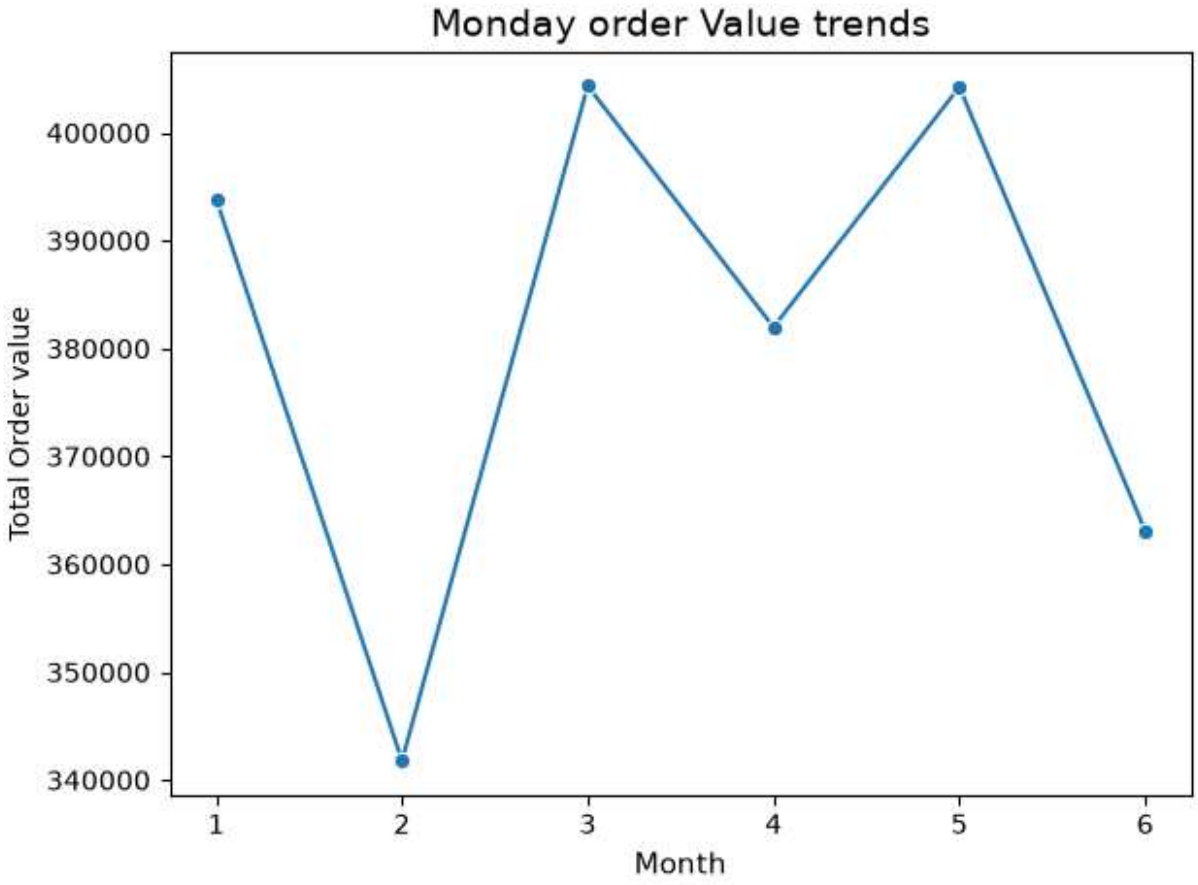
In [39]:
```
plt.figure(figsize=(10,2))
sns.histplot(x="day",data=df,bins=31,kde=True)# kde shows  avg line
plt.title("order Distribution by Day of month")
plt.xlabel("Day of Month",fontsize=11)
plt.ylabel("Number of Orders",fontsize=11)
plt.tight_layout()
plt.show()
```



In [40]:
```
df["Month"]=df["order_date"].dt.month
```

In [46]:
```
data= df.groupby("Month")["order_value_inr"].sum().reset_index()
sns.lineplot(x="Month",y="order_value_inr",data=data,marker="o")
plt.title("Monday order Value trends",fontsize=14)
plt.xlabel("Month",fontsize=11)
```

```
plt.ylabel("Total Order value",fontsize=10)
plt.xticks(range(1,7))
plt.tight_layout()
plt.show()
```



7. Conclusion

In this Exploratory Data Analysis (EDA) project, I analyzed food delivery order data using Python, Pandas, Matplotlib, and Seaborn.

3 3-are-there-any-null-values

EDA The analysis explored order distribution across cities, vendor-wise revenue, the relationship between weather conditions and delivery delays, delivery time categories, delivery distance versus delivery time, daily order distribution, and monthly order value trends. This project helped me understand data cleaning, data transformation, data aggregation, and data visualization to identify meaningful patterns in the dataset. Tools Used: Python, Pandas, Matplotlib, Seaborn Project Outcome: Developed practical skills in data analysis and visualization to support data-driven decision-making. If you'd like, let me know if you need this text formatted into a specific template (like a resume bullet point or project summary) or if you need help analyzing the content further!

In [ ]: